NTU Physics Discovery Camp
Computational Nonlinear Lab: Chaos with Logistic Map

Prof. Chew Lock Yue, Wang Yuhao, and Cheong Jian Wei

June 2025

1. Introduction

The logistic map is a very simple mathematical model used to describe the growth of biological populations. The biologist Robert May had used this model to illustrate that simple models can give rise to bewildering complex behavior and chaos.

The model is built up as follows. Consider a particular biological species (e.g. fruit flies, ants, etc.) and we are concerned with its population each year. Within one year, individuals in the population are born or died away. If we know there are N_0 individuals in year 0, we would like to find out the number N_1 of individuals in year 1.

In the simplest case, we might assume

$$N_1 = AN_0, \tag{1}$$

where A is a parameter that depends on the conditions of the environment, such as food supply, water, general weather conditions etc., assuming that A remains the same for subsequent generations (i.e., year 1, year 2, ..., year n, etc.).

Question 1*: What happen to the population if (a) $A < 1$ and (b) $A > 1$?*

- (a) $N_\infty \to 0$
- (b) $N_\infty \to \infty$

We know that if the population grows too large, there will not be enough food to support the larger population, or perhaps the individual will more easily be eaten up by predators. Hence, the population growth will be limited.

This feature can be incorporated into the model by adding another term that is unimportant for small N, but becomes more important when N increases. One way of doing this is to introduce a term proportional to N^2 as follows:

$$N_{n+1} = AN_n - BN_n^2. \tag{2}$$

If $B \ll A$, then the second term in the above equation will not be important until N gets sufficiently large. The minus sign means that the second term tends to decrease the population. However, we do not expect N_{n+1} to become negative. Hence, there is some maximum population number, which is

$$N_{\max} = \frac{A}{B}, \tag{3}$$

beyond which $N_{n+1} < 0$.

***Question 2**: Obtain $N_{\max} = \frac{A}{B}$.*

$$\begin{aligned} N_{n+1} = AN_n - BN_n^2 &> 0 \\ AN_n &> BN_n^2 \\ A &> BN_n \\ \frac{A}{B} &> N_n. \end{aligned}$$

Therefore, for $N_{n+1} > 0$, the upper bound of N_n is $\frac{A}{B}$ or $N_{\max} = \frac{A}{B}$.

Let us next normalize the equation $N_{n+1} = AN_n - BN_n^2$ by $N_{\max}$ and denote $x_n = \frac{N}{N_{\max}}$. We arrive, after some algebraic manipulation, the logistic map equation:

$$x_{n+1} = Ax_n(1 - x_n) \tag{4}$$

with $x_n \in [0, 1]$. Note that x_n is the population (as a fraction of $N_{\max}$) in the n-th year. This equation plays a very important role in the development of the theory of chaos.

***Question 3**: Show the derivation of $x_{n+1} = Ax_n(1 - x_n)$.*

Normalizing $N_{n+1} = AN_n - BN_n^2$ with $N_{\max}$, and letting $x_n = N_n/N_{\max}$,

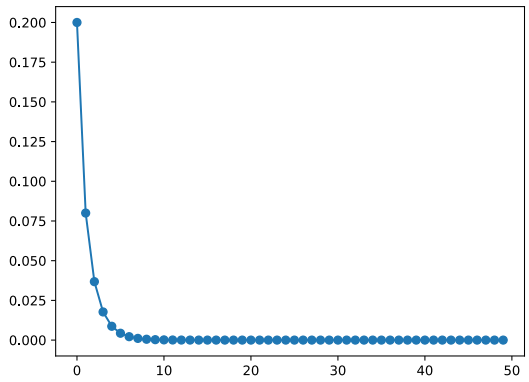
$$\begin{aligned} \frac{N_{n+1}}{N_{\max}} &= A\frac{N_n}{N_{\max}} - B\frac{N_n^2}{N_{\max}} \\ x_{n+1} &= Ax_n - Bx_nN_n \\ &= Ax_n\left(1 - \frac{B}{A}N_n\right) \\ &= Ax_n\left(1 - \frac{1}{N_{\max}}N_n\right) \\ &= Ax_n(1 - x_n). \end{aligned}$$

2. Fixed Points and Stability

Next, we shall adjust the parameter A and observe how this affect the population dynamics. We shall restrict A to the range $0 \leq A \leq 4$ so that Eq. 4 will map the interval $0 \leq x_n \leq 1$ to itself.

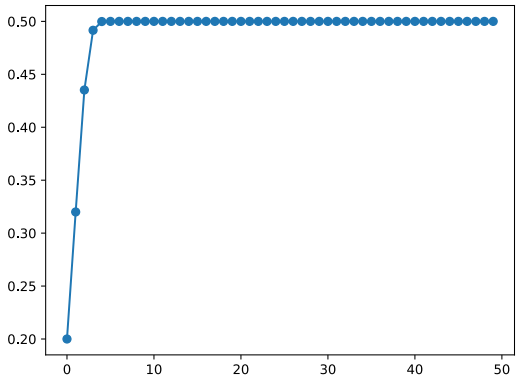
***Computation 1**: Iterate $x_{n+1} = Ax_n(1 - x_n)$ with $A = 0.5$ and $A = 2$ at different n. Observe and comment on what happen as n becomes large for the two cases.*

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.zeros(50)  # Initialize x_0 to x_49. This also set them to be 0
x[0] = 0.2        # Set x_0 = 0.2

A = 0.5
for n in np.arange(0, 49, step=1):  # iterate n over [0, 49)
    x[n+1] = A * x[n] * (1 - x[n])  # The logistic map

plt.plot(x, 'o-')
```



We can do the same for the case of $A = 2$.

```python
x = np.zeros(50)
x[0] = 0.2

A = 2
for n in np.arange(0, 49, step=1):
    x[n+1] = A * x[n] * (1 - x[n])

plt.plot(x, 'o-')
```

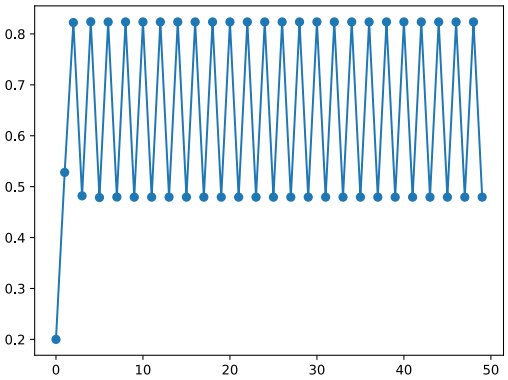


Observes that x_n stabilized at a fixed value as n becomes large.

***Illustration**: Explain the concept of fixed point and its stability for the respective cases of $A = 0.5$ and $A = 2$.*

***Computation 2**: Iterate the logistic map $x_{n+1} = Ax_n(1 - x_n)$ with $A = 3.3$. Observe and comment on what happen as n becomes large.*

```python
x = np.zeros(50)
x[0] = 0.2

A = 3.3
for n in np.arange(0, 49, step=1):
    x[n+1] = A * x[n] * (1 - x[n])

plt.plot(x, 'o-')
```



Observes that x_n alternates between two values.

Tip

We can check the values that x_n oscillates between with

```
1  x[-1], x[-2]
```

```
(0.47942701982423314, 0.8236032832060693)
```

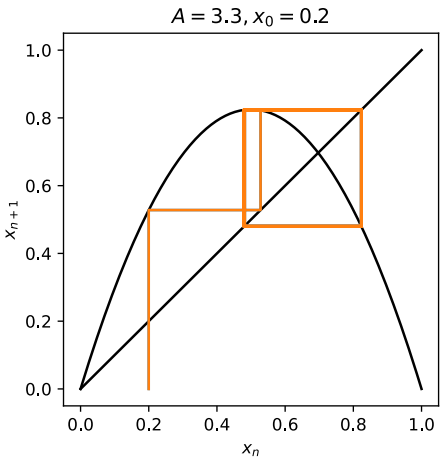
Note that the `[-1]` and `[-2]` indices refers to the last and second last indices which in this case is x_{49} and x_{48}.

We can also check different ranges of values with

```
1  x[10:20], x[:15], x[40:]
```

```
(array([0.82359529, 0.47944409, 0.8236056 , 0.47942207, 0.82360261,
        0.47942846, 0.82360348, 0.4794266 , 0.82360323, 0.47942714]),
 array([0.2       , 0.528     , 0.8224128 , 0.48196496, 0.82392663,
        0.47873607, 0.82350789, 0.47963073, 0.82363081, 0.47936823,
        0.82359529, 0.47944409, 0.8236056 , 0.47942207, 0.82360261]),
 array([0.82360328, 0.47942702, 0.82360328, 0.47942702, 0.82360328,
        0.47942702, 0.82360328, 0.47942702, 0.82360328, 0.47942702]))
```

***Illustration**: Explain on the second-iterate map and period-2 cycle. Demonstrate with the cobweb diagram.*



An attractor of a dynamical system is the set of points at which the system eventually settles into. Let us next examine the orbit diagram of the logistic map, which is a diagram of the system's attractor as a function of A.

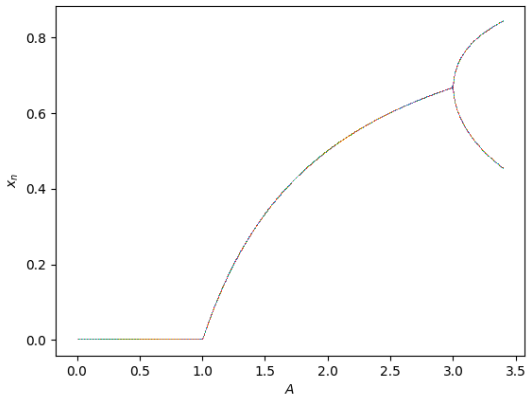
Earlier, we observed that for different values of A, x_n reached different fixed points or cycles. The orbit diagram is simply a plot of these fixed points and cycles against A.

To generate the orbit diagram,

1. First, loop through the range of values of A.
2. Then, for each value of A in the loop, iterate the logistic map for a large number of cycles, say from x_0 to x_{599}, to allow the system to settle down to its eventual behavior.
3. Next, plot many values of x_n against the corresponding A for values of n after it settles down to its eventual behavior, say $x_{300}, ..., x_{599}$.

***Computation 3**: Plot the orbit diagram for the range $0 \leq A \leq 3.4$.*

```python
for A in np.linspace(0, 3.4, 3000):  # loop through many values of A

    x = np.zeros(600)  # Initialize x_0 to x_599
    x[0] = 0.2         # Set x_0 = 0.2

    # Iterate over the logistic map from x_1 up till x_599
    for n in np.arange(0, 599, step=1):
        x[n+1] = A * x[n] * (1 - x[n])

    # Plot x_300 to x_599 at A
    # we plot with the symbol ','
    plt.plot(np.repeat(A, 300), x[300:], ',')  # plot(x values, y values)

plt.xlabel("$A$")
plt.ylabel("$x_n$")
```



Tip

```python
np.repeat(0.5, 10)
```

```
array([0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5])
```

We want to plot x_{300} to x_{599} as our y-values on the plot, and so we need to specify their corresponding x-values, since each point on a plot requires a coordinate (x, y).

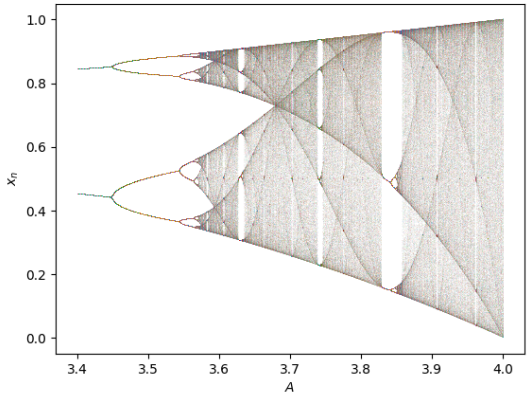
In this case, we want to plot all of them above the same A, i.e., $(A, x_{300}), (A, x_{301}), ..., (A, x_{599})$ which is why `np.repeat(A, 300)` was used to specify the x-values to be A.

3. Bifurcation Diagram

***Computation 4**: Plot the orbit diagram for the range $3.4 \leq A \leq 4$.*

```python
for A in np.linspace(3.4, 4, 3000):  # loop through many values of A
```

```
2
3       x = np.zeros(600)  # Initialize x_0 to x_599
4       x[0] = 0.2         # Set x_0 = 0.2
5
6       # Iterate over the logistic map from x_1 up till x_599
7       for n in np.arange(0, 599, step=1):
8           x[n+1] = A * x[n] * (1 - x[n])
9
10      # Plot x_300 to x_599 at A
11      # note that alpha sets the transparency of the points
12      plt.plot(np.repeat(A, 300), x[300:], ',', alpha=0.05)
13
14  plt.xlabel("$A$")
15  plt.ylabel("$x_n$")
```



Observe the presence of a period-doubling in the plotted bifurcation diagram. The bifurcation can be examined in greater detail through the following computation.

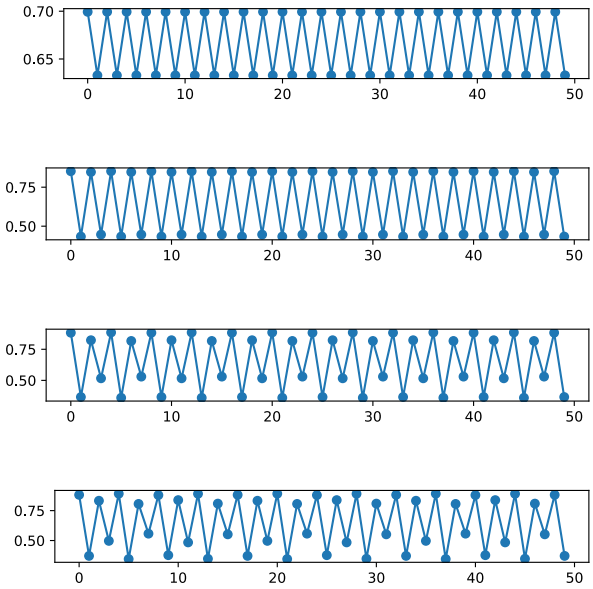
***Computation 5**: Compute the steady state dynamics of the logistic map for $A_1 = 3.01$, $A_2 = 3.45$, $A_3 = 3.545$ and $A_4 = 3.5655$. Infer the period of the orbit in each case.*

We simply do the same as above for these specific values of A. Alternatively, one can of course just change the values of A like in ***Computation 1*** and ***Computation 2***.

Python

```
1   for A in [3.01, 3.45, 3.545, 3.5655]:
2
3       x = np.zeros(600)
4       x[0] = 0.2
5
6       for n in np.arange(0, 599, step=1):
7           x[n+1] = A * x[n] * (1 - x[n])
8
9       # We create a new figure for each value of A
10      plt.figure(figsize=(6, 1))
11      plt.plot(x[-50:], '.-')  # We plot only the last 50 values
12      print(x[-20:])           # We also print out the last 20 values
```

```
[0.69937714 0.63284875 0.69937714 0.63284876 0.69937713 0.63284876
 0.69937713 0.63284877 0.69937713 0.63284877 0.69937712 0.63284877
 0.69937712 0.63284878 0.69937712 0.63284878 0.69937712 0.63284878
 0.69937711 0.63284879]
[0.84732753 0.44630437 0.85255289 0.43368729 0.84732905 0.44630072
 0.85255154 0.43369058 0.84733056 0.44629712 0.8525502  0.43369383
 0.84733205 0.44629355 0.85254888 0.43369704 0.84733352 0.44629003
 0.85254758 0.43370021]
[0.8168705  0.5303075  0.88299376 0.36625444 0.82283749 0.51677577
 0.88525234 0.36010334 0.81687054 0.5303074  0.88299378 0.36625438
 0.82283743 0.51677589 0.88525233 0.36010338 0.81687058 0.53030731
 0.8829938  0.36625433]
[0.80863719 0.55173647 0.88183136 0.37154226 0.83253929 0.49709353
 0.89134488 0.34531577 0.80606253 0.55737939 0.87963597 0.37750277
 0.83787264 0.48434494 0.89050116 0.34766777 0.80863719 0.55173647
 0.88183136 0.37154226]
```



We also print the values as it is difficult to determine the period of the orbit from the plots alone. We can infer the period of the orbit by counting the number of cycles before a value is repeated again (note that the values might not be exact due to numerical errors).

***Question 4**: Evaluate:*

$$\delta_3 = \frac{A_3 - A_2}{A_4 - A_3}. \tag{5}$$

We simply substitute the values for A_2, A_3, and A_4 in:

$$\delta_3 = \frac{A_3 - A_2}{A_4 - A_3} = \frac{3.545 - 3.45}{3.5655 - 3.545} = \frac{0.095}{0.0205} \approx 4.634$$

It is important to note that the successive bifurcations come faster and faster. Ultimately, the A_m converge to a limiting value of A_∞. The convergence is essentially geometric: in the limit of large m, the distance between transitions shrinks by a constant factor

$$\delta = \lim_{m\to\infty} \frac{A_m - A_{m-1}}{A_{m+1} - A_m} = 4.669... \tag{6}$$

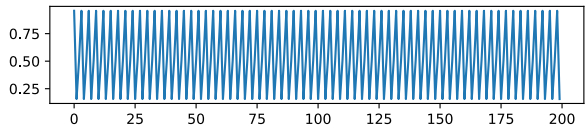
Interestingly, this convergence factor δ is universal.

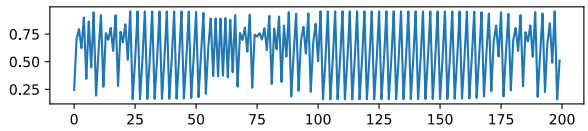
4. Route to Chaos

At A_∞, the dynamics of the logistic map is chaotic. The manner in which chaos is reached as A approaches A_∞ from below is known as the *period-doubling route to chaos*. Let us go beyond A_∞ and examine the dynamics of logistic map there.

***Computation 6**: Compute the steady state dynamics of the logistic map for $A = 3.83$ and $A = 3.8282$.*

```python
for A in [3.83, 3.8282]:

    x = np.zeros(600)
    x[0] = 0.2

    for n in np.arange(0, 599, step=1):
        x[n+1] = A * x[n] * (1 - x[n])

    plt.figure(figsize=(6, 1))
    plt.plot(x[-200:], '-')
```



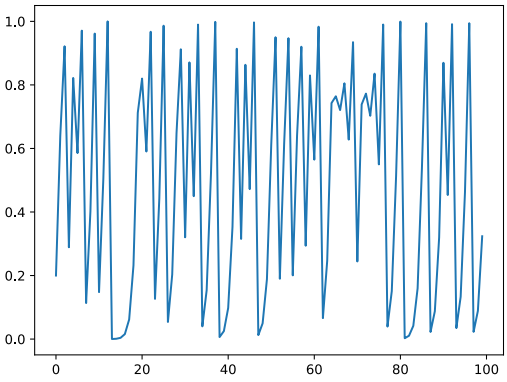


As A_∞ is approached from above, we observe the *intermittency route to chaos*.

5. Chaotic Dynamics of the Logistic Map at $A = 4$

***Computation 7**: Iterate the dynamics of the logistic map at $A = 4$ for a particular initial condition x_0. Observe the aperiodic dynamics.*

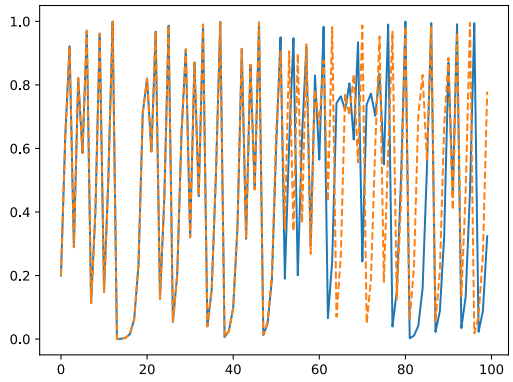
```python
x = np.zeros(600)
x[0] = 0.2

A = 4
for n in np.arange(0, 599, step=1):
    x[n+1] = A * x[n] * (1 - x[n])

plt.plot(x[:100], '-')  # We plot just the last 100 values
```



***Computation 8**: We now run the iteration from a slightly perturbed initial condition $x_0 + \epsilon$ with ϵ a very small number and compare the dynamics. Notice the fast divergence of the results even though ϵ is small.*

By putting the case of $x_0 + \epsilon$ in the same code block as ***Computation 5***, we can plot both cases in the same plot to see their differences.

```python
# Case of x_0 = 0.2
x = np.zeros(600)
x[0] = 0.2

A = 4
for n in np.arange(0, 599, step=1):
    x[n+1] = A * x[n] * (1 - x[n])

plt.plot(x[:100], '-')

# Case of x_0 = 0.2 + 10^-16
x = np.zeros(600)
x[0] = 0.2 + 1e-16

A = 4
for n in np.arange(0, 599, step=1):
    x[n+1] = A * x[n] * (1 - x[n])

plt.plot(x[:100], '--')
```



We observe that even though the initial condition x_0 changes by an extremely small amount of just 10^{-16}, its effects can be large.

The trademark of chaos is its sensitive dependence on initial condition.